



\$identifiers

Document number:

P4234

Date:

2026-05-12

Audience:

SG22

EWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Matthias Wippich <mfwippich@gmail.com>

Co-authors:

Murat Can Çağrı <cancagri.dev@gmail.com>



ABSTRACT: This paper proposes making \$ in identifiers conditionally-supported.

0.1 R0 May 2026

1 Introduction

2 Proposed change

2.1 Why not allow it unconditionally?

3 Additional Motivation

3.1 Linker and toolchain symbol conventions

3.1.1 Linker conventions

3.1.2 Symbol patching

3.2 Stricter Domains

3.3 Additional identifier namespace

3.3.1 Macros

3.3.2 Generated code

4 Wording

4.1 [lex.name] Identifiers

5 Acknowledgements

6 References

§ Revision history



§ 0.1. R0 May 2026

Original version of the paper.

§ 1. Introduction

One of the oldest and most widely supported extensions to C++ is allowing \$ in identifiers. This feature's origins predate both standard C++ and standard C. As GCC notes in its [documentation](#):

”

In GNU C, you may normally use dollar signs in identifier names. This is because many traditional C implementations allow such identifiers.

Amongst many other compilers this is supported by [MSVC](#), [GCC](#) (going back to at least [GCC 1.27!](#)), Clang, EDG (might need opt-in), icx and nvc++ (see [Compiler Explorer](#)). Some of these implementations make it configurable, for example GCC and Clang both have `-f[no-]dollars-in-identifiers` to enable or disable this feature (which defaults to enable).

In recent years, C has made quite a few changes to what they consider an identifier. The original rules allowed any implementation-defined characters, the revised ones did not (see [\[WG14 N2836\]](#)). To address this, [\[WG14 N3145\]](#) introduced a carve-out to allow \$ anywhere in identifiers as an implementation extension.

In C++ however, the situation is different. The C++ standard does not acknowledge \$ in identifiers at all. As a result, the use of \$ in identifiers pedantically renders your code ill-formed. In practice, however, almost all implementations support it as a non-standard extension.

§ 2. Proposed change

To align with C and implementation reality, this paper proposes making the use of \$ in identifiers conditionally-supported. This would not make \$ in identifiers portable, but it would at least make them explicitly recognized by the standard when supported by the implementation.

[\[WG14 N3145\]](#) mentions that during discussion it was noted that allowing \$ in identifiers is a massive syntactic land-grab. However, due to the popularity of this extension, it seems unrealistic that we can ever use \$ for anything that would conflict with `$identifiers`.

§ 2.1. Why not allow it unconditionally?

GCC notes in its [documentation](#):



” However, dollar signs in identifiers are not supported on a few target machines, typically because the target assembler does not allow them.

Examples of such targets are [RS/6000](#), [AVR](#), and [MMIX](#).

In theory, we could mandate \$ to be valid in identifiers and ask compilers to work around such problematic targets in some way or another. However, that's a rather big ask - especially given the popularity and history of this extension, making stronger support guarantees than C *and* asking for more than what implementations have been supporting for decades does not seem wise.

§ 3. Additional Motivation

§ 3.1. Linker and toolchain symbol conventions

§ 3.1.1. Linker conventions

Some embedded toolchains expose identifiers containing \$ through linker-defined symbols and custom linker conventions.

One such example is armlink from the [ARM MDK toolchain](#):

” Symbols that contain the character sequence \$\$, and all other external names containing the sequence \$\$, are names reserved by ARM.

Unfortunately the examples given in armlink's documentation are not valid C++.

```
// oops, none of these are valid identifiers
extern int Image$$ER_ZI$$Base;
extern int Image$$ER_ZI$$Limit;
extern int Image$$ER_RO$$Base;
extern int Image$$ER_RO$$Limit;
extern int Image$$ER_RW$$Base;
extern int Image$$ER_RW$$Limit;
```

With GCC and Clang this can also be addressed by explicitly specifying the name used in assembler code ([Compiler Explorer](#))

```
extern int Image_ER_ZI_Base asm("Image$$ER_ZI$$Base");
extern int Image_ER_ZI_Limit asm("Image$$ER_ZI$$Limit");
extern int Image_ER_RO_Base asm("Image$$ER_RO$$Base");
extern int Image_ER_RO_Limit asm("Image$$ER_RW$$Limit");
extern int Image_ER_RW_Base asm("Image$$ER_RW$$Base");
extern int Image_ER_RW_Limit asm("Image$$ER_RW$$Limit");
```

However, it seems much cleaner and overall less error-prone (did you notice the typo?) to avoid repetition and instead actually allow such implementations to support dollars in identifiers directly. 

Likewise, if you target RISC-V you may have come across `__global_pointer$` before (8.5.1 of [\[RISC-V ELF Specification\]](#)). The same argument as before applies - however, in this case it will still result in a reserved identifier due to the double underscore.

§ 3.1.2. Symbol patching

Furthermore, armlink supports `$Sub$$` and `$Super$$` symbol patterns to allow [patching existing definitions](#). This feature is commonly used for startup hooks, C library retargeting and other situations where an existing symbol cannot be modified. [For example](#), RP2040 MDK headers define the following wrapper macros:

```
# define WRAPPER_FUNC(__FUNC)    $Sub$$$__FUNC
# define REAL_FUNC(__FUNC)       $Super$$$ __FUNC
```

As with linker-defined `Image$$...` symbols, there are still some portability concerns. It does however show that `$` containing identifiers are a common part of documented embedded toolchain practice rather than merely accidental parser behavior.

§ 3.2. Stricter Domains

In some domains (such as defense, aerospace and avionics) that deal with safety or mission-critical systems, conformance to the C++ International Standard may be a contractual requirement. For example, the requirements for the [\[TRITON\]](#) project, as specified in NATO IFB-CO-13859-TRITON (Unclassified), state:

”

5.5.6 TRITON software will be developed with any variations from the languages or specifications given below as "Preferred Languages":

C++ [ISO/IEC 14882]

[T1-R1791] TRITON shall comply with the standards and language specifications given in the Description as the Preferred Languages. Any variations from the languages or specifications shall be agreed with the Purchaser

In such environments, using a compiler extension is not merely an engineering portability concern. It can become a compliance issue. Deviations from the contracted obligations may need to be recorded in project coding guidelines, reviewed by QA, justified to the customer or main contractor, and in some cases handled through contract amendments or formal deviation processes. In such stricter domains, the bureaucracy and paperwork required for such changes can easily take months.

As mentioned before, making the use of `$` in identifiers conditionally-supported would not make such identifiers portable, nor would it require every implementation to accept them. However, it would change the status of the construct from an unstandardized implementation

extension into a construct explicitly recognized by the C++ Standard, when supported by the implementation.



This distinction is useful for stricter environments. A project could specify that it uses C++ as defined by ISO/IEC 14882, together with the conditionally-supported use of \$ in identifiers on implementations that document support for it. This is easier to audit, easier to encode in project guidelines, and easier to justify than relying on undocumented or folklore implementation behavior.

§ 3.3. Additional identifier namespace

§ 3.3.1. Macros

Another somewhat common use is in macro identifiers. [GitHub search](#) finds more than 5600 uses in code recognized as C++.

For example consider

```
#if MSVC
#   define $inline_never __declspec(noinline)
#   define $inline_always __forceinline
#else
#   define $inline_never [[gnu::noinline]]
#   define $inline_always [[gnu::always_inline]] inline
#endif

#define $inline(_opt)      $inline_##_opt
#define $template         template for (auto _ : "")
#define $pi               3
#define $assert(...)      \
    do {                  \
        ((__VA_ARGS__) ? (void)0 : throw std::runtime_error(#__VA_ARGS__)); \
    } while (false)

$inline(always) int foo(int a) {
    $assert(a > 0);
    return a * $pi;
}

$inline(never) int bar(int, char) {
    $template {
        constexpr auto ctx = std::meta::current_function();
        static constexpr auto [... Idx] = std::make_index_sequence<parameters_o
        return (foo([:variable_of(parameters_of(ctx)[Idx]):]) + ... + 0);
    }
}
```

As noted earlier, using \$ in any identifiers that survive until the assembler takes over can be problematic with some targets. Since macros are long gone at that point, using dollars in macros is relatively safe. This essentially gives us another identifier namespace that is visually distinct from regular identifiers.

If used consistently only for macros, this has the benefit of making them visually stand out. We can validate consistent use easily by using a custom clang-tidy check (see [example implementation](#)). 

Not only does this reduce the chances of name collisions, it also helps when surveying code - after all, macros come with sharp edges.

§ 3.3.2. Generated code

In a similar fashion, dollars in identifiers can be useful for generated code.

While there has always been C++ code generators, with the recent adoption of [P2996] and its limited reification features such as `define_aggregate`, there is an ever increasing need for ways to avoid name collisions stemming from generated/reified code.

As a result of that, it's more than likely that we'll continue to see all sorts of creative mangling/uglification schemes - (conditionally) allowing the use of \$ provides another viable alternative.

For example, consider a simple struct to struct of arrays transformation (adopted from the struct to struct of arrays example in [P2996]):

```
template <typename T, size_t N>
struct struct_of_arrays_impl {
    struct impl;

    consteval {
        auto ctx = std::meta::access_context::current();
        for (std::meta::info member : nonstatic_data_members_of(^T, ctx)) {
            auto array_type = substitute(^std::array, {type_of(member)}, std::meta::
            auto mem_descr = data_member_spec(array_type, {.name = identifier_of(me
            new_members.push_back(mem_descr);
        }

        define_aggregate(^impl, new_members);
    }
};

template <typename T, size_t N>
using struct_of_arrays = struct_of_arrays_impl<T, N>::impl;
```

If we, for some reason, attempt to inherit from this generated type, we might run into collisions.

```
struct Particle {
    vec3 position;
    vec3 velocity;
    float lifetime;
};

template <size_t N>
struct Particles : struct_of_arrays<Particle, N> {
```

```

vec3 position(size_t i) const; // oops, collides with generated position m
vec3 velocity(size_t i) const; // also collides
float lifetime(size_t i) const; // also collides
};

```



While this is arguably a naming issue (that could and probably should be `get_position` instead), these sorts of problems can generally be avoided by uglifying identifiers in generated code. Conditionally allowing dollars gives users a few more options to make those generated names more unique and less likely to collide with user code.

§ 4. Wording

Make the following changes to the C++ Working Draft. All wording is relative to [\[N5032\]](#), the latest draft at the time of writing.

§ **[lex.name]** Identifiers

Add a new paragraph after [\[lex.name\]](#) paragraph 1, add \$ to *nondigit* with optional remark.

”

Identifiers

[lex.name]

identifier:

identifier-start

identifier identifier-continue

identifier-start:

nondigit

an element of the translation character set with the Unicode property

identifier-continue:

digit

nondigit

an element of the translation character set with the Unicode property

nondigit: one of

a b c d e f g h i j k l m

n o p q r s t u v w x y z

A B C D E F G H I J K L M

N O P Q R S T U V W X Y Z

– \$

digit: one of

0 1 2 3 4 5 6 7 8 9

[Note: The character properties `XID_Start` and `XID_Continue` are described by UAX #44 of the Unicode Standard. — end note]

1 The program is ill-formed if an *identifier* does not conform to Normalization Form C as specified in the Unicode Standard.



[Note: Identifiers are case-sensitive. — *end note*]

[Note: [uaxid] compares the requirements of UAX #31 of the Unicode Standard with the C++ rules for identifiers. — *end note*]

[Note: In translation phase 4, *identifier* also includes those preprocessing-tokens ([lex.pptoken]) differentiated as keywords ([lex.key]) in the later translation phase 7 ([lex.token]). — *end note*]



2 The inclusion of \$ in *nondigit* is conditionally-supported.

[Note: Use of \$ anywhere in an identifier is allowed if the implementation supports it. — *end note*]

3 The identifiers in Table 4 have a special meaning when appearing in a certain context. [...]

§ 5. Acknowledgements

Thanks to Jan Schultke for the markup language and document generator used for this paper and thanks to all the awesome people who gave feedback on this proposal.

§ 6. References

[N5032] Thomas Köppe. Working Draft Programming Languages — C++ (2025-12-15)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5032.pdf>

[WG14 N3145] \$ in Identifiers v2
<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3145.pdf>

[P2996] Reflection for C++26
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2996r13.html>

[TRITON] NATO TRITON IFB-CO-13859-TRITON
https://www.gdaee.mil.gr/wp-content/uploads/2021/09/5_2.pdf

[JSFAVRules] Lockheed Martin. Joint Strike Fighter Air Vehicle C++ Coding Standards for the System Development and Demonstration Program
<https://www.stroustrup.com/JSF-AV-rules.pdf>

[MISRACPP2023] The MISRA Consortium. MISRA C++:2023: Guidelines for the use of C++17 in critical systems
<https://misra.org.uk/product/misra-cpp2023/>

[AUTOSARCPP14] *AUTOSAR*. Guidelines for the use of the C++14 language in critical and safety-related systems

https://www.autosar.org/fileadmin/standards/R17-03_R1.1.0/AP/AUTOSAR_RS_CPP14Guidelines.pdf

[WG14 N2836] Identifier Syntax using Unicode Standard Annex 31

<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2836.pdf>

[RISC-V ELF Specification] RISC-V ABIs Specification

<https://lists.riscv.org/g/tech-psabi/attachment/61/0/riscv-abi.pdf>

